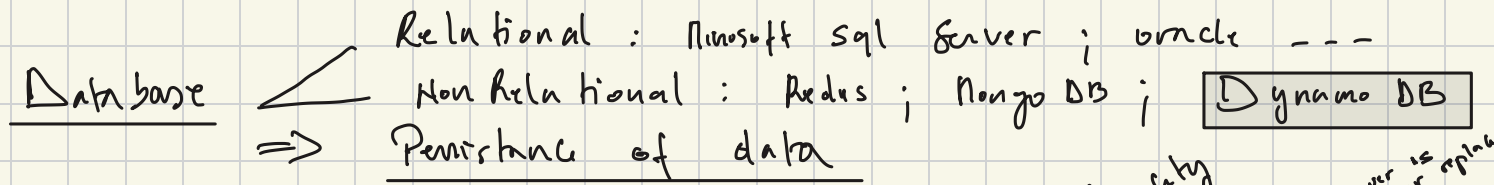
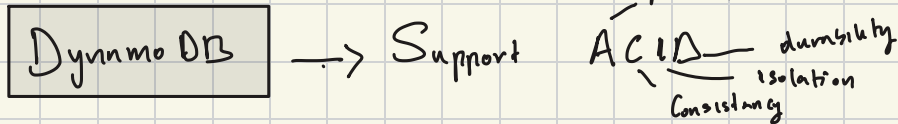
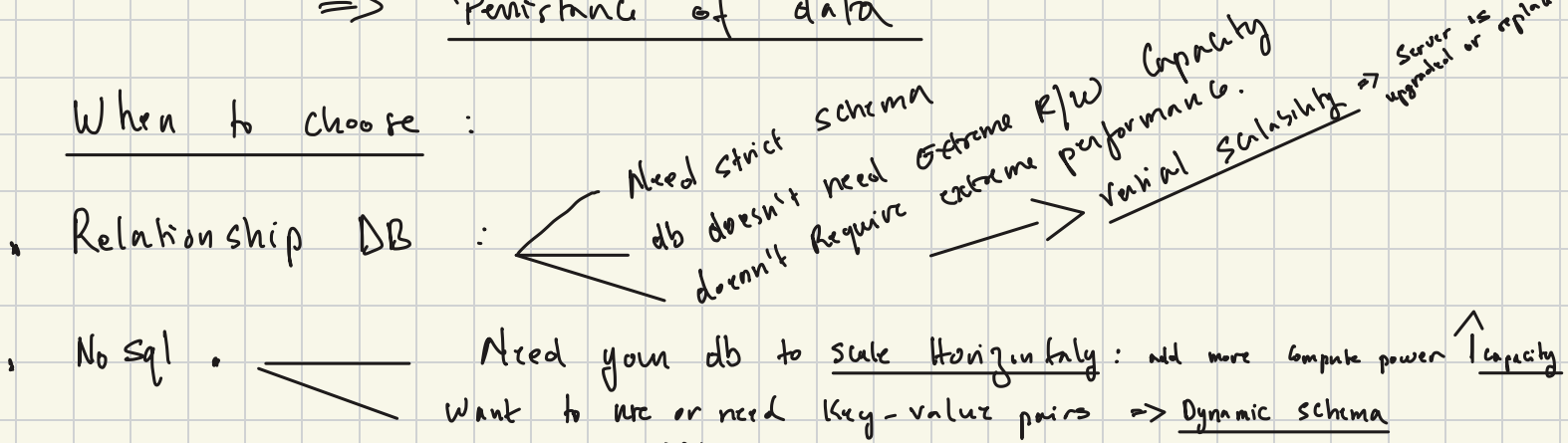




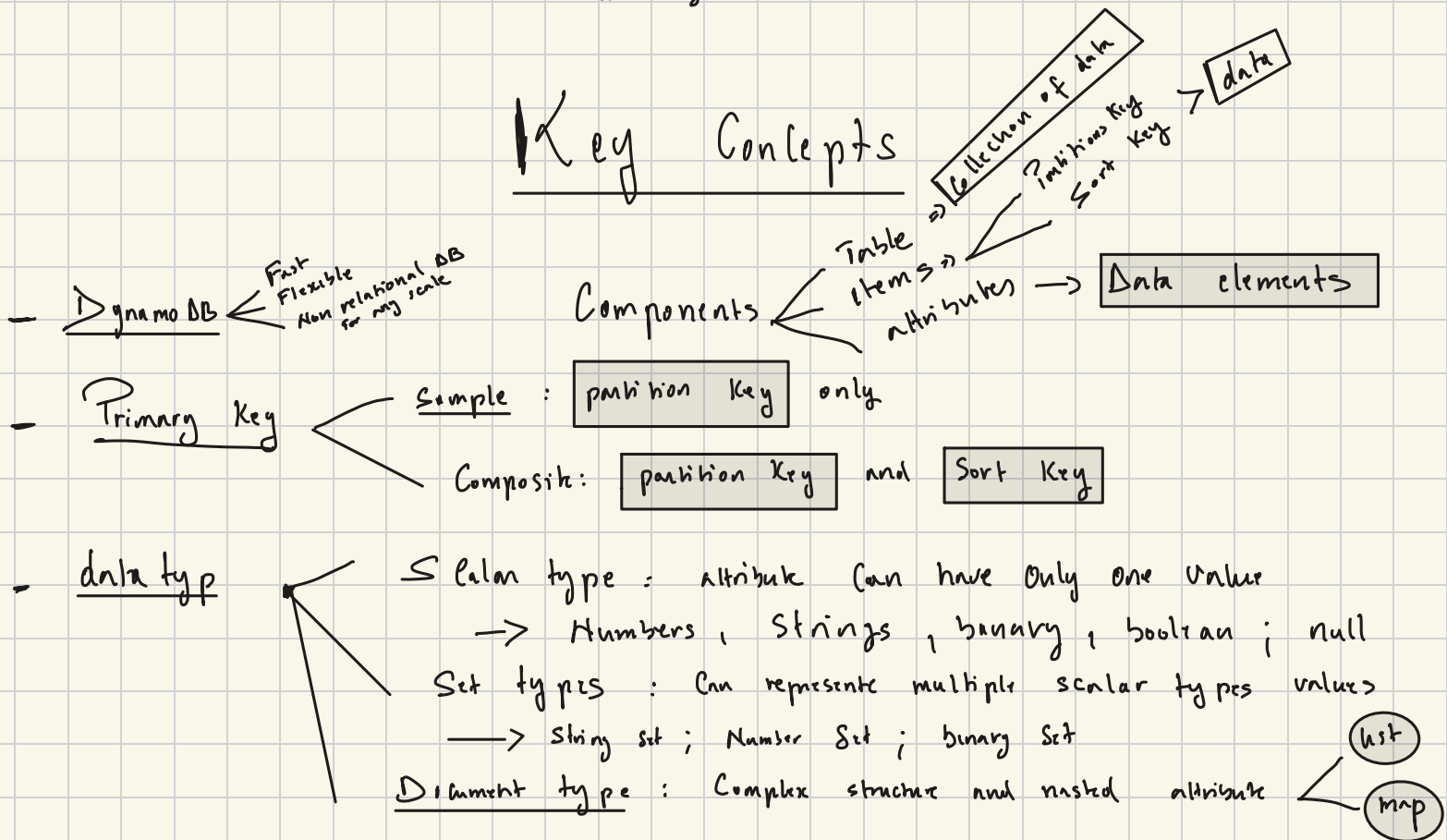
Dynamo DB



When to choose :



Key Concepts



Partition and data distribution

→ Partition : Allocation of storage

→ retrieve or write data

⇒ data are store based on the PK

Partition Key-value ⇒ If only partition-key
(call it also Hash attribute)

Partition and Sort Key :

- Store data with same PK closely. and use Sort-key to retrieve data.

Secondary indexes

⇒ Use to query data based on non-primary-key attributes ; It defines an alternative key.

⇒ Sans index ⇒ on ne peut chercher que par la clé primaire ; avec avec les index → on peut chercher avec d'autre attribut ⇒ chercher avec un attribut ≠ clé primaire

2 types

- Global Secondary index ⇒ Filtre au delà de la partition
- Local Secondary index ⇒ Filtre dans la même partition
↳ identique au S.K ⇒ LSI
ajoute des filtre supplémentaire

GSI

- Eventually consistency uniquement
- PK et SK différent
- Throughput séparé (own RCU/WCU)
- Pas de limite de taille — création à tout moment
- Max 20 par table

L.S.I

- même PK que la table ; SK diff.
- Partage le throughput de la table
- limite 10GB par valeur PK
- Max 5 par table
- Crée à la création de la table uniquement.
- Support le strong consistency read.

→ un LSI ti les données sur une partition → et la partition stock jusqu'à 10GB. et faudrait alors utiliser le GSI
⇒ à y penser au design.

Read Consistency

→ les données sont copiées sur plusieurs serveurs ⇒ 1 Sec de synchronisation partout
⇒ décalage ou non dans la lecture.

• Eventually Consistent Read

0.1 RCU pour 4 KB ⇒ Cheap.

• Strongly Consistent Read

1 RCU pour 4 Kb ⇒ Expensive.

4 opérations ⇒ ACID

a Cost $\begin{cases} \text{Provisioned} \rightarrow \text{Fixed R/W capacity} \\ \text{On-demand} \rightarrow \text{Dynamically manage.} \end{cases}$

RCU $\xrightarrow{\text{Read Capacity Unit}} 1 \text{ lecture / seconde pour 4 Kb.}$
WCU $\xrightarrow{\text{Write}} 1 \text{ écriture / s pour 1 Kb}$

Calcul : Read / Write throughput

→ 3 infos nécessaires $\begin{cases} \text{Combien d'ops par seconde?} \\ \text{taille moyenne des items?} \\ \text{Quel type de Consistency?} \end{cases} \rightarrow \text{Provisioned}$

RCU : $\begin{cases} \text{S. Const.} \Rightarrow \text{RCU} = [\text{taille} / 4] \times \text{lecture/s} \\ \text{E. Const.} \Rightarrow \text{RCU} = [\text{taille} / 4] \times \text{lecture/s} / 2 \end{cases}$

WCU : $[\text{taille} / 1] \times \text{écritures/s}$

In production

⇒ Demander on-demand → run after 2 weeks → CloudWatch → RCU/s and WCU/s and Size ⇒ Provisioned
↳ Demand Explode? → erreur 400 ⇒ Auto scaling or on-demand

Stream and global table

Stream ⇒ Flux qui capture toute les modifications dans l'ordre chronologique stocké pendant **24 h** Insert
update
Delete > Sur la table.

→ modification occur Dynamo ⇒ Stream record.

4 types { Keys only → eli uniquement.
New image → item après modif.
old image → item avant modif.
New and old image → both

↳ Contient le changement et une autre app peut le lire et réagir.

Global table ⇒ table dynamo répliquée automatiquement dans plusieurs régions AWS.
chaque région ⇒ copie identique ⇒ à travers le stream.

⇒ En cas de conflict ⇒ **Last Writer Wins**

⇒ Strongly consistent ⇒ même région uniquement

⇒ Update : propagé en quelques secondes / Transactions désactivé par défaut.

⇒ The global table offer a solution to deploy a multi-Region, multi-active database without needing to build and maintain our own replication solution.

→ Specify the regions where we want the table to be available.

→ One replicant (single table) per region.

Backup and Restore

On demand : We can write full backup to meet requirements for long-term data retention.

⇒ backup & restore. ↳ impact availability

⇒ On demand backup is asynchronous, applying all changes. | no limit of the number of backup demand.
doesn't affect table throughput

point-in-time recovery : Protect again accidental write or delete operations ⇒ don't need to schedule backup. ⇒ table can be restored to any point in time during the last

35 days

BASIC operations for dynamoDB tables

- Control operations: Create and manage DynamoDB tables
- Data Operations: CRUD actions on data in a table
- Batch operations: Get and write batches of items in a DynamoDB table
- Transaction operations: Make coordinated, all-or-nothing changes to multiple items both within and across tables

tablename
Key Schema - PK
Attribut de partition
Provisioned Throughput.

- ⇒ Delete Item: Supprime un item par clé primaire.
- ⇒ Conditions Expressions: S'applique au CRUD → marche une fois la condition remplie.
- ⇒ GetItem: Lit un seul item par clé primaire complète. ⇒ éventuellement contextualisé by default.
 - ↳ doit fournir PK et SK si Composite
- ⇒ Query: Lit tous les items d'un même PK. ⇒ plus efficace que Scan
 - ↳ obligatoire de spécifier l'index ou la table.
- ⇒ Scan: Lit tous les items de la table ou index
 - ↳ Expensive. ⇒ éviter en prod.